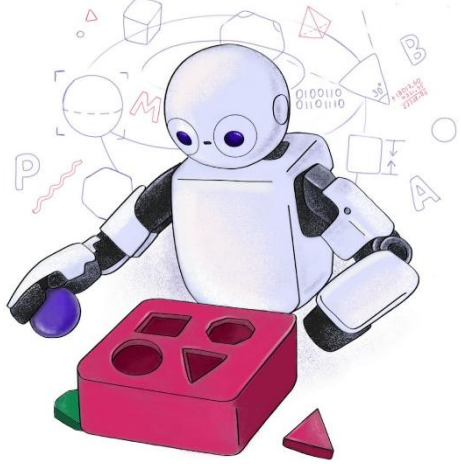
T320 - Introdução ao Aprendizado de Máquina II: *Classificação (Parte IV)*



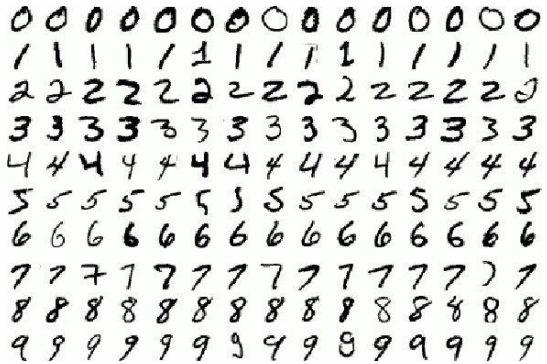
Inatel

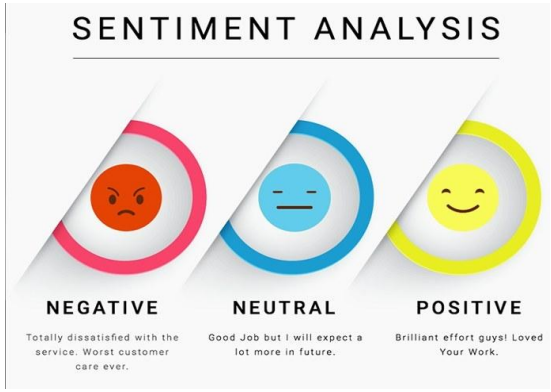
Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

Recapitulando

- Anteriormente, aprendemos como o ***regressor logístico*** funciona e pode ser usado em problemas de classificação binária, após a discretização do valor de saída.
- Na sequência, veremos como lidar com problemas de classificação que envolvem mais de 2 classes.

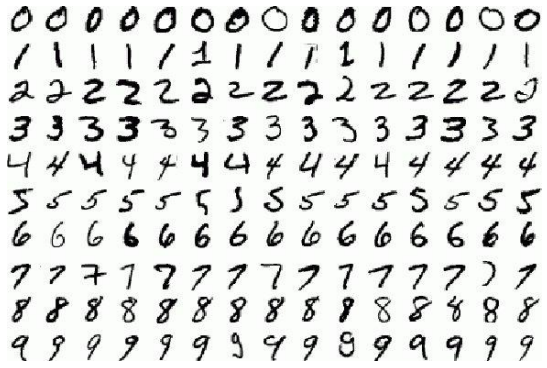
Casos multi-classe

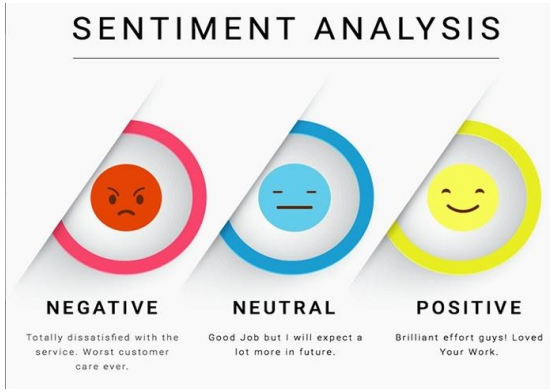




- Até agora, vimos problemas com 2 classes (i.e., $Q = 2$).
- Porém, e quando o problema possuir mais de 2 classes (i.e., $Q > 2$)?
- Por exemplo
  - Reconhecimento de dígitos escritos à mão: 10 dígitos.
  - Classificação de texto: Esportes, Economia, Política, Entretenimento, etc.
  - Classificação de sentimentos: Neutro, Positivo, Negativo.

Casos multi-classe





- Quando $Q > 2$, chamamos o problema de classificação ***multiclasses***.
- Algumas abordagens para a ***classificação multiclasse*** são:
  - Um-Contra-o-Resto
  - Um-Contra-Um
  - Regressão Softmax
- As duas primeiras usam ***classificadores binários***, como o ***regressor logístico***.
- A terceira abordagem é uma generalização do ***classificador logístico*** para problemas multiclasse.

Um-Contra-o-Resto

- Nesta abordagem, treina-se ***um classificador binário*** **por classe** q, para predizer a probabilidade $P\left(C_q \middle| \boldsymbol{x}(i); \boldsymbol{a}\right), \forall q \in \{1, \dots, Q\}$, onde C_q é a classe positiva e as demais formam a classe negativa.
- Em outras palavras, treina-se Q ***classificadores binários***, onde para cada classificador, a classe positiva é a q-ésima classe e a classe negativa é a junção de todas as outras, $Q - 1$, classes.
- Fazendo isso, nós ***transformamos um problema com Q classes em Q problemas binários***.
- Cada um dos Q classificadores binários é representado pela função hipótese $h_{\boldsymbol{a}}^{q}\left(\boldsymbol{x}(i)\right)$.

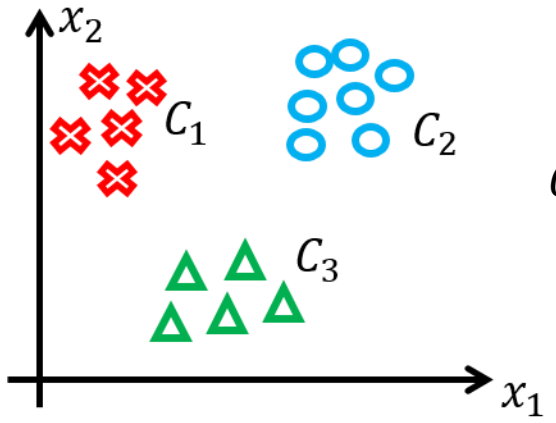
Um-Contra-o-Resto

- Portanto, o q-ésimo *classificador* deve indicar a classe positiva caso o exemplo pertença à q-ésima classe ou indicar a classe negativa caso o exemplo pertença a qualquer uma das outras $Q - 1$ classes.
- Após o treinamento, ***para cada exemplo*** de entrada, $\boldsymbol{x}(i)$, ***realiza-se Q predições*** e escolhe-se a classe que maximiza $h_{\boldsymbol{a}}^{q}\big(\boldsymbol{x}(i)\big)$

$$C_q = \arg\max_{q} h_{\boldsymbol{a}}^{q}\big(\boldsymbol{x}(i)\big).$$

- A ***vantagem*** desta abordagem é que treina-se apenas Q ***classificadores***.
- Uma ***desvantagem*** é que cada ***classificador binário*** precisa ser treinado com um conjunto negativo que é Q-1 vezes maior, o que pode ***tornar o treinamento lento*** e ***aumentar a possibilidade de classes desbalanceadas***.

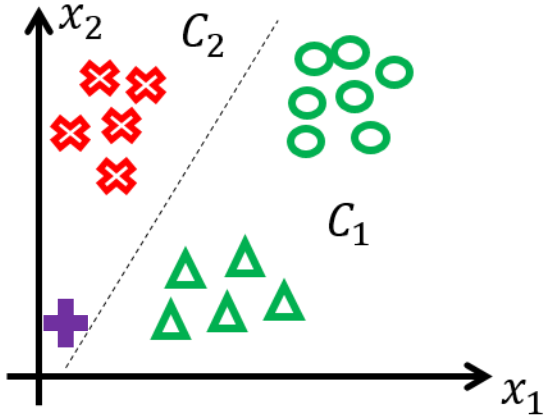
Um-Contra-o-Resto



$$Q = 3$$

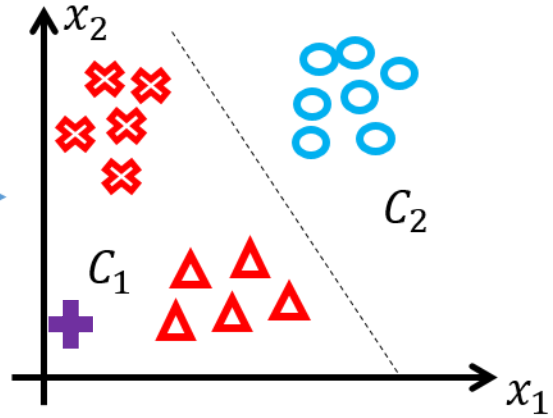
Passamos a ter ***3*** ***classificadores binários***.

$C_1(+)/C_2\&C_3(-)$



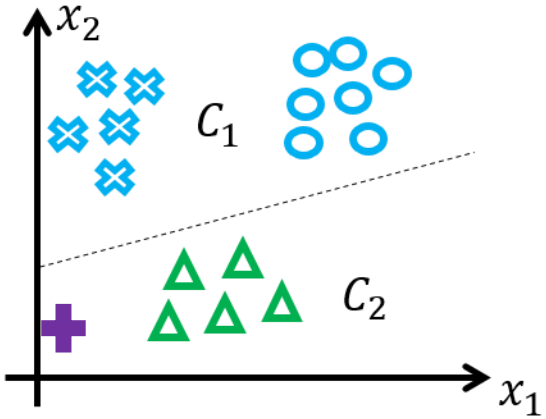
$h_a^1(\boldsymbol{x}(i)) = 0.55$

$C_2(+)\ /C_1\&C_3(-)$



$h_a^2(\boldsymbol{x}(i)) = 0.05$

$C_3(+)\ /C_1\&C_2(-)$



$h_a^3(\boldsymbol{x}(i)) = 0.85$

A qual classe ✚ pertence?

Fronteira de decisão

✚ Exemplo de validação

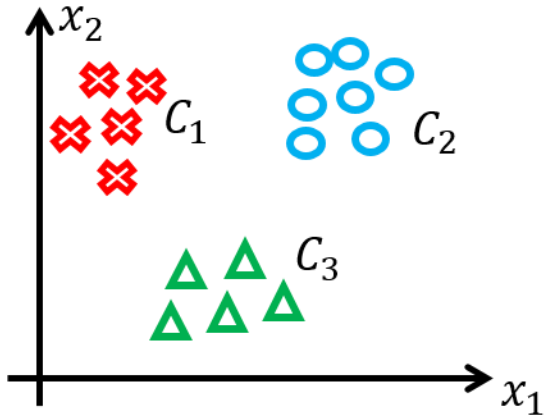
Um-Contra-Um

- Nesta abordagem, treina-se um classificador binário para **cada um dos possíveis pares de classes (sem repetição)**.
- Ao todo, treina-se $Q(Q-1)/2$ ***classificadores binários***.
  - Por exemplo, se $Q = 4$, então treina-se 6 ***classificadores*** para classificar entre C_1/C_2, C_1/C_3, C_1/C_4, C_2/C_3, C_2/C_4, e C_3/C_4.
- ***Transformamos um problema com Q classes em $Q(Q-1)/2$ problemas binários.***
- No final, cada exemplo é classificado conforme o ***voto majoritário*** entre os ***classificadores***.
- Ou seja, a classe que receber mais votos é a classe atribuída ao exemplo.

Um-Contra-Um

- A principal ***vantagem*** dessa abordagem é que cada classificador binário precisa ser ***treinado apenas com as duas classes*** que ele deve distinguir, portanto, a ***chance de desbalanceamento é menor***.
- Além disso, o ***tempo de treinamento de cada classificador também é menor***, pois treina-se cada um deles com pares de classes.
- A ***desvantagem*** é que, quanto maior o número de classes Q, maior pode ser o ***tempo total de treinamento***.
  - Por exemplo, se $Q = 10$, temos que treinar 45 classificadores.

Um-Contra-Um

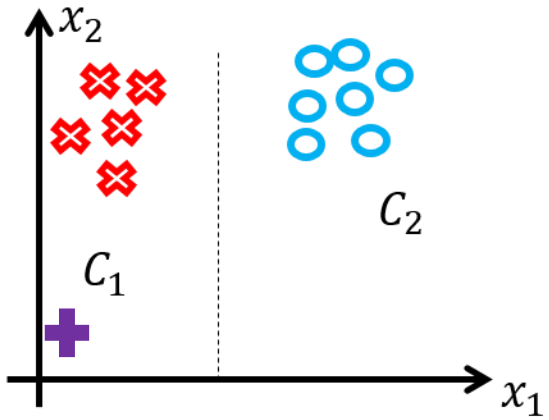


$$Q = 3$$

$$\frac{Q(Q-1)}{2} = 3$$

Passamos a ter ***3*** ***classificadores binários***.

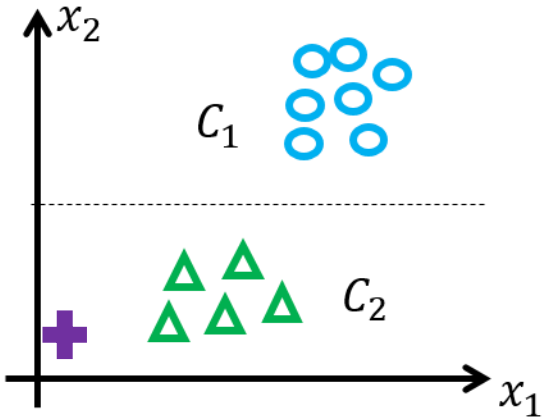
$C_2(+)/C_1(-)$



$h_a^1(\boldsymbol{x}(i))$ = 0.05 (voto em C_1)

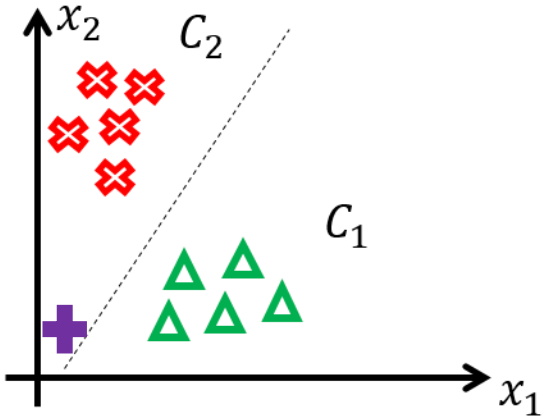
A qual classe ✚ pertence?

$C_3(+)/C_2(-)$



$h_a^2(\boldsymbol{x}(i))$ = 0.85 (voto em C_3)

$C_1(+)/C_3(-)$



$h_a^3(\boldsymbol{x}(i))$ = 0.55 (voto em C_1)

Fronteira de decisão

✚ Exemplo de validação

Regressor *softmax*

- Também conhecida como **regressão logística multinomial**.
  - Pois suas saídas podem ser interpretadas como as **probabilidades de uma variável categórica** (as classes) **dado os atributos e os pesos.**
  - Ou seja, a regressão ***softmax* estima a probabilidade de um exemplo de entrada pertencer a cada uma das Q possíveis classes**.
- O regressor *softmax* tem Q saídas, onde cada saída representa a **probabilidade** de um exemplo pertencer a uma das Q classes.
  - Por exemplo, para um problema com 4 classes, teremos um classificador com 4 saídas.

Regressor *softmax*

- Ele prediz **apenas uma classe por classificação**, portanto, ele deve ser usado apenas com **classes mutuamente exclusivas** como, por exemplo, diferentes tipos de plantas, dígitos, carros, etc.
  - **Classes mutuamente exclusivas**: caso onde os exemplos pertencem a apenas uma das Q classes.
  - Por exemplo, notícias, músicas e imagens, por exemplo, podem pertencer a várias classes ao mesmo tempo.
- O **regressor *softmax*** **possui uma função hipótese de classificação**, $h_{\boldsymbol{a}}^{q}\big(\boldsymbol{x}(n)\big)$, e, consequentemente, **uma função discriminante**, $g_q(\boldsymbol{x}(n))$, **para cada classe** q.
  - Se temos 3 classes, teremos 3 funções discriminantes, $g_q(\boldsymbol{x}(n))$.

Regressor *softmax*

- A **função hipótese de classificação** associada à q-ésima classe, $h_{\boldsymbol{a}}^{q}\big(\boldsymbol{x}(n)\big)$, é obtida passando-se a **função discriminante** da q-ésima classe, $g_q(\boldsymbol{x}(n))$, através da **função (de ativação) *softmax***,

$$P\big(C_q \mid \boldsymbol{x}(n), \boldsymbol{a}_q\big) = h_{\boldsymbol{a}}^{q}\big(\boldsymbol{x}(n)\big) = \frac{e^{g_q(\boldsymbol{x}(n))}}{\sum_{j=0}^{Q-1} e^{g_j(\boldsymbol{x}(n))}} \in \mathbb{R}\,[0,1],$$

onde $\boldsymbol{a}_q = \begin{bmatrix} a_0^q & a_1^q & \cdots & a_K^q \end{bmatrix}^{\boldsymbol{T}} \in \mathbb{R}^{K+1\times 1}$ é o **vetor de pesos** da q-ésima **função discriminante**, $g_q(\boldsymbol{x}(n))$, e n indica o número da amostra.

- Cada função discriminante tem seu próprio vetor de pesos, $\boldsymbol{a}_q$.
- Além do regressor *softmax*, ela é usada na **camada de saída** de **redes neurais** empregadas em tarefas de **classificação.**

Função de ativação *softmax*

- Ela transforma os **valores das Q funções discriminantes em uma distribuição de probabilidades**.
  - O somatório de termos exponenciais no denominador **normaliza** o valor da q-ésima saída de tal forma que o somatório das Q saídas seja igual a 1

$$P\left(C_q \mid \boldsymbol{x}(n), \boldsymbol{a}_q\right) = h_{\boldsymbol{a}}^{q}\left(\boldsymbol{x}(n)\right) = \frac{e^{g_q(\boldsymbol{x}(n))}}{\boxed{\sum_{j=0}^{Q-1} e^{g_j(\boldsymbol{x}(n))}}} \in \mathbb{R}\,[0,1]\ \forall q.$$

- Dado um exemplo de entrada, ela **atribui** uma **probabilidade para cada classe** possível.
- É uma generalização da função sigmoide.

Função de ativação *softmax*

Ativações da camada de saída

Função de ativação *softmax*

Probabilidades

$$\begin{array}{c} g_1(\boldsymbol{x}) \\ g_2(\boldsymbol{x}) \\ g_3(\boldsymbol{x}) \\ g_4(\boldsymbol{x}) \\ g_5(\boldsymbol{x}) \end{array} \begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \longrightarrow \boxed{\frac{e^{g_i(\boldsymbol{x})}}{\sum_{j=1}^{Q} e^{g_j(\boldsymbol{x})}}} \longrightarrow \begin{bmatrix} 0.01 \\ 0.9 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

$Q = 5$ (número de classes)

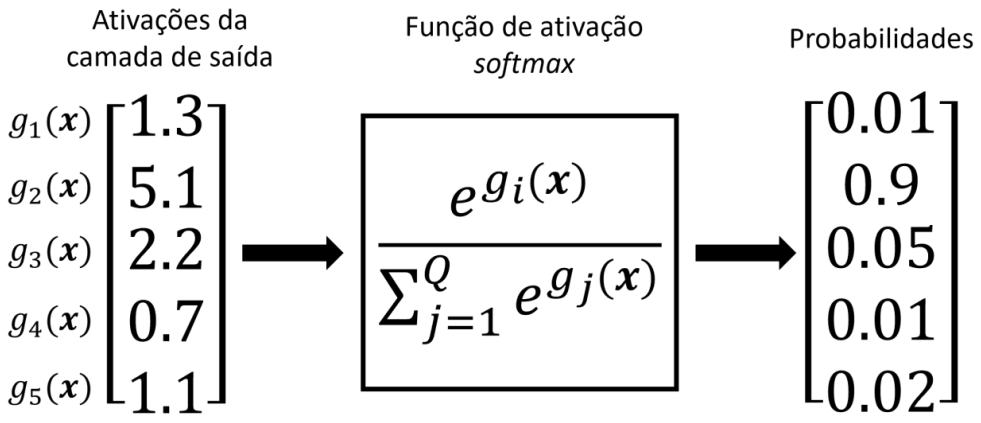
- A função *softmax* opera da seguinte maneira.
- Dado um **vetor com valores de funções discriminantes**, $g_q(\boldsymbol{x}(n)), \forall q$, ela
  - calcula a exponencial de cada função discriminante,
  - normaliza essas exponenciais dividindo-as pela soma de todas as exponenciais, gerando uma distribuição de probabilidades das classes.

OBS.: *e* é a base do logaritmo natural.

Função de ativação *softmax*

- Ela **garante que as saídas estejam no intervalo entre 0 e 1 e a soma de todas as saídas seja igual a 1**, tornando-as interpretáveis como uma **função massa de probabilidade multinomial**.
- Ela atribui uma **probabilidade condicional**, $P\left(C_q \mid \boldsymbol{x}(n), \boldsymbol{a}_q\right) \in [0, 1]$, a cada classe, q, onde **a soma das Q probabilidades deve ser igual a 1**

$$P(C_0 \mid \boldsymbol{x}(n), \boldsymbol{a}_0) + P(C_1 \mid \boldsymbol{x}(n), \boldsymbol{a}_1) + \cdots + P\left(C_{Q-1} \mid \boldsymbol{x}(n), \boldsymbol{a}_{Q-1}\right) = 1.$$



$Q = 5$ (número de classes)

Função de ativação *softmax*

Ativações da camada de saída

Função de ativação *softmax*

Probabilidades

$$\begin{array}{c} g_1(\boldsymbol{x}) \\ g_2(\boldsymbol{x}) \\ g_3(\boldsymbol{x}) \\ g_4(\boldsymbol{x}) \\ g_5(\boldsymbol{x}) \end{array} \begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \longrightarrow \boxed{\frac{e^{g_i(\boldsymbol{x})}}{\sum_{j=1}^{Q} e^{g_j(\boldsymbol{x})}}} \longrightarrow \begin{bmatrix} 0.01 \\ 0.9 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

- Assim como com o regressor logístico, podemos usar **hiperplanos** ou **polinômios** como **funções discriminantes**.
- Nosso objetivo é encontrar um **modelo** (i.e., **equação** e os respectivos **pesos** das Q funções hipótese) que **atribua uma alta probabilidade para a classe alvo** e, consequentemente, **uma baixa probabilidade para as demais classes**.

Função de erro

- Para atingir o objetivo, devemos definir uma **função de erro** a ser **minimizada**.
- Usamos a **generalização da função da entropia cruzada**, chamada de **entropia cruzada categórica**, a qual é dada por

$$J_e(\boldsymbol{A}) = -\frac{1}{N}\sum_{n=0}^{N-1}\sum_{q=0}^{Q-1} 1\{y(n) == q\}\log\left(\hat{y}\,(n)\right),$$

onde 1{·} é a **função indicadora** (1{condição verdadeira} = 1 e 1{condição falsa} = 0), $\boldsymbol{A} \in \mathbb{R}^{K+1\times Q}$ é a matriz com os **pesos** das Q **funções hipótese**, $y(n)$ é rótulo do n-ésimo exemplo e $\hat{y}\,(n) = h_{\boldsymbol{a}}^{q}\left(\boldsymbol{x}(n)\right)$ é a q-ésima saída do modelo para o n-ésimo exemplo.

- **OBS**.: O erro tende a 0 quando $h_{\boldsymbol{a}}^{q}(\boldsymbol{x}(n))$ tende a 1, caso contrário, o erro aumenta.

Função de erro

$$J_e(\boldsymbol{A}) = -\frac{1}{N}\sum_{n=0}^{N-1}\sum_{q=0}^{Q-1} 1\{y(n) == q\}\log\left(\hat{y}\,(n)\right)$$

- A matriz $\boldsymbol{A}$ contém em suas colunas os vetores de pesos, $\boldsymbol{a}_q$, de cada uma das Q **funções discriminantes**.
- Essa função mede a **diferença entre as distribuições de probabilidade preditas pelo modelo e as distribuições dos rótulos** das classes.

Função de erro

- Usando a codificação ***one-hot***, a equação pode ser re-escrita como

$$J_e(\boldsymbol{A}) = -\frac{1}{N}\sum_{n=0}^{N-1} \boldsymbol{y}(n)^T \log\left(\widehat{\boldsymbol{y}}\,(n)\right),$$

onde $\boldsymbol{y}(n) = [1\{y(n) == 0\}, \cdots, 1\{y(n) == Q-1\}]^T \in \mathbb{R}^{Q\times 1}$ é um vetor com os rótulos usando a codificação ***one-hot*** e $\widehat{\boldsymbol{y}}\,(n) = \left[\hat{y}_0(n), \cdots, \hat{y}_{Q-1}(n)\right]^T \in \mathbb{R}^{Q\times 1}$ é um vetor com as Q saídas do modelo para o n-ésimo exemplo de entrada.

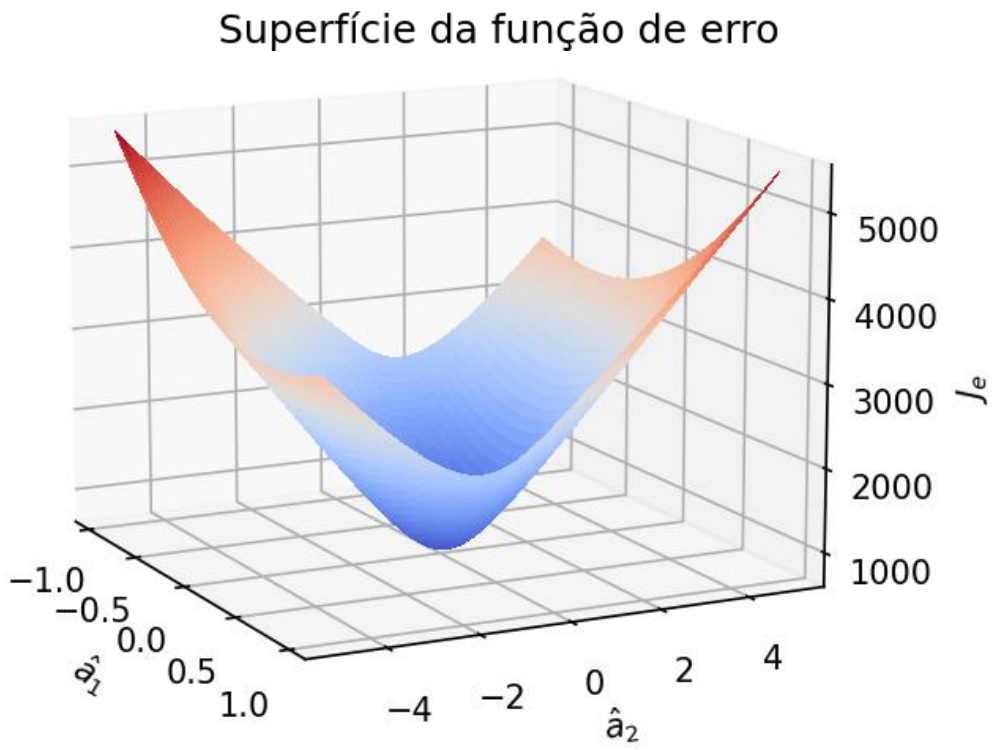
- Durante o **treinamento** do modelo, o **objetivo é minimizar a entropia cruzada categórica**, ou seja, fazer com que as **probabilidades previstas pelo modelo se aproximem ao máximo dos rótulos dos exemplos**.

Função de erro

$$J_e(\boldsymbol{A}) = -\frac{1}{N}\sum_{n=0}^{N-1} \boldsymbol{y}(n)^T \log\left(\widehat{\boldsymbol{y}}\,(n)\right).$$

- Notem que quando existem apenas duas classes ($Q = 2$), a **função de erro médio** é equivalente à **função de erro médio** do **regressor logístico**.
- Ou seja, mesmo tendo sido pensado para o caso onde $Q > 2$, o regressor *softmax* também pode ser usado quando $Q = 2$.
- Entretanto, existe um problema com essa função de erro.

Função de erro



- Essa **função de erro** **não é linear** e, portanto, **não existe uma forma fechada** para encontramos os pesos.
- Porém, ela é **convexa** e **contínua**.
- Assim, **o algoritmo do gradiente descendente (qualquer versão) pode ser usado para encontrar o mínimo global**.

Gradiente descendente

- A atualização dos **pesos** da q-ésima função discriminante, $g_q(\boldsymbol{x}(n))$, é dada por

$$\boldsymbol{a}_q = \boldsymbol{a}_q - \alpha \frac{\partial J_e(\boldsymbol{A})}{\partial \boldsymbol{a}_q}, \forall q.$$

- Considerando o **hiperplano** como **função discriminante**, a derivada da **função de erro**, $J_e(\boldsymbol{A})$, com respeito ao vetor de pesos, $\boldsymbol{a}_q$, é dada por

$$\frac{\partial J_e(\boldsymbol{A})}{\partial \boldsymbol{a}_q} = -\frac{1}{N}\sum_{n=0}^{N-1}\left[1\{y(n) == q\} - h_{\boldsymbol{a}}^q\big(\boldsymbol{x}(n)\big)\right]\boldsymbol{x}(n) = -\frac{1}{N}\boldsymbol{X}^T\big(\boldsymbol{y}_q - \widehat{\boldsymbol{y}}_q\big),$$

onde $\boldsymbol{y}_q$ e $\widehat{\boldsymbol{y}}_q \in \mathbb{R}^{N\times 1}$são vetores contendo os q-ésimos elementos dos vetores *one-hot* e de saída do regressor *softmax* para cada um dos N vetores de atributo, respectivamente.

Regressor *softmax*

$$\frac{\partial J_e(\boldsymbol{A})}{\partial \boldsymbol{a}_q} = -\frac{1}{N}\boldsymbol{X}^T\left(\boldsymbol{y}_q - \widehat{\boldsymbol{y}}_q\right).$$

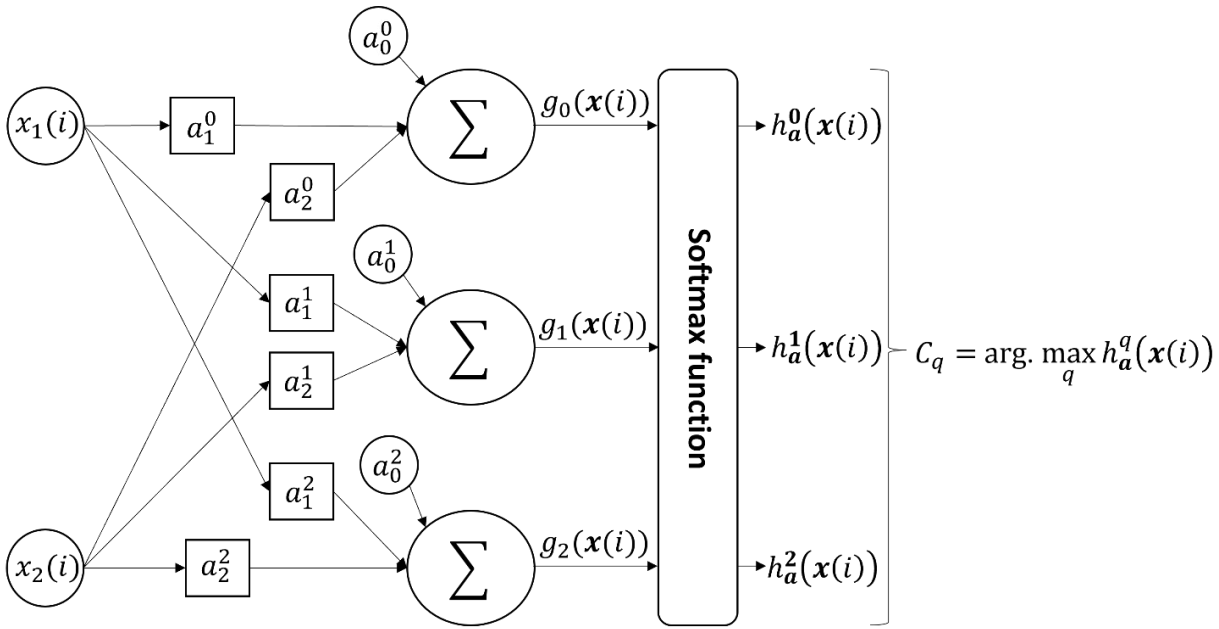
- Notem que essa expressão do vetor gradiente é **idêntica** àquela obtida para o caso da **regressão logística**.
- O vetor gradiente pode ser reescrito **levando-se em consideração todos os Q vetores de peso**

$$\frac{\partial J_e(\boldsymbol{A})}{\partial \boldsymbol{A}} = -\frac{1}{N}\boldsymbol{X}^T\left(\boldsymbol{Y} - \widehat{\boldsymbol{Y}}\right),$$

onde $\boldsymbol{Y}$ e $\widehat{\boldsymbol{Y}} \in \mathbb{R}^{N\times Q}$ são matrizes contendendo em cada linha os vetores $\boldsymbol{y}(n)^T$ e $\widehat{\boldsymbol{y}}(n)^T$ para todos os N exemplos, respectivamente.

- Esse resultado é diretamente estendido para polinômios.

Regressor *softmax*

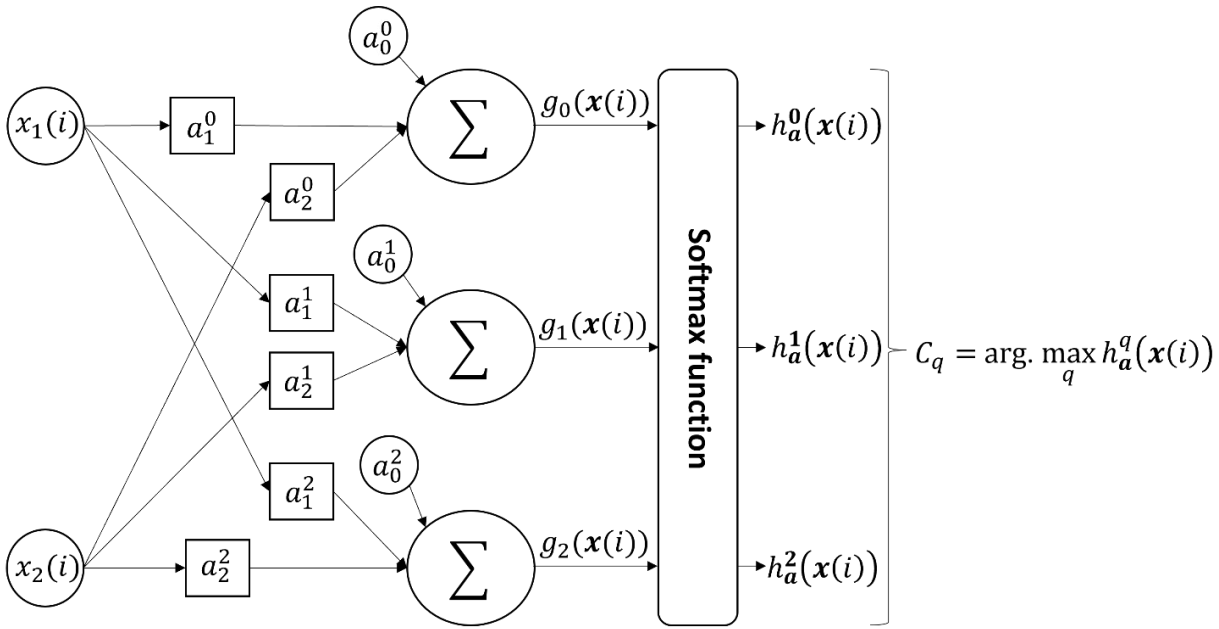


- Após o treinamento, durante a **fase de inferência**, o classificador atribui ao exemplo de entrada, $\boldsymbol{x}(n)$, a classe, q, com a **maior probabilidade estimada**, ou seja, atribui a classe, q, da função hipótese com maior valor:

$$C_q = \arg.\ \max_q h_{\boldsymbol{a}}^q\big(\boldsymbol{x}(n)\big).$$

- A arquitetura de um **regressor *softmax*** para um problema com três classes (i.e., $Q = 3$) e dois atributos (x_1 e x_2) é mostrada ao lado.

Regressor *softmax*



- Em resumo, a ideia por trás da **regressão *softmax*** é bastante simples:
  - dado um exemplo de entrada, $\boldsymbol{x}(n)$,
  - o regressor *softmax* calcula uma "**pontuação**", para cada classe q, i.e., as saídas das Q funções discriminantes $g_q(\boldsymbol{x}(n)), \forall q$.
  - e, em seguida, estima a probabilidade de cada classe aplicando a função *softmax* às "**pontuações**".

Exemplo: softmax_regressor_with_scikit_learn.ipynb

Tarefas

- **Quiz**: *"T320 - Quiz - Classificação (Parte IV)"* que se encontra no MS Teams.
- **Exercício Prático**: **Laboratório #4**.
  - Pode ser acessado através do link acima (Google Colab) ou no GitHub.
  - Se atentem aos prazos de entrega.
  - Instruções para resolução e entrega dos laboratórios.

Obrigado!